



J D College Of Engineering & Management

Unit-1

Basic concepts of Testing

Dr. A. R. Welekar

Associate Professor(CSE)

❖ Need of Testing:

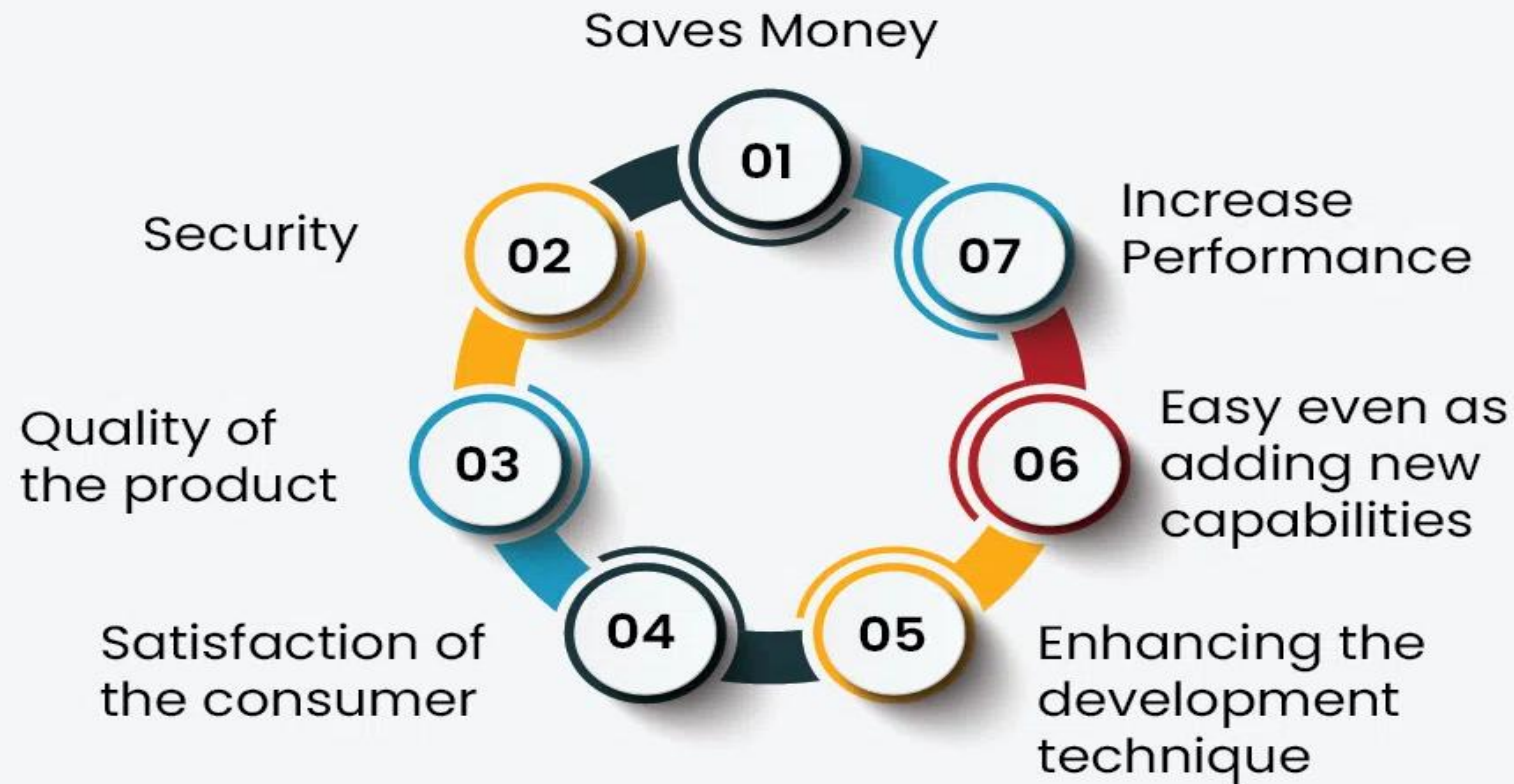
Software testing is essential to ensure that applications work correctly, perform well, and are free from critical bugs before reaching users. It helps identify issues early, improves software quality, and enhances user experience.

What is Software Testing:

- It is a method to assess the functionality of the software program.
- The process checks whether the actual software matches the expected requirements and ensures the software is bug-free.
- The purpose of software testing is to identify the errors, faults, or missing requirements in contrast to actual requirements.
- It mainly aims at measuring the specification, functionality, and performance of a software program or application.

❖ Need of Testing:

why Software Testing is really important?



❖ **Basic concepts- Bug, Errors, Faults, Defects & Failures:**

- The main purpose of software testing is to identify errors, deficiencies, or missing requirements with respect to actual requirements.
- Software Testing is Important because if there are any bugs or errors in the software, they can be identified early and can be solved before the delivery of the software product.
- Let us see difference between errors, faults, defects & failures

❑ What is a Bug:

A bug refers to defects which means that the software product or the application is not working as per the adhered requirements set. When we have any type of logical error, it causes our code to break, which results in a bug. It is now that the Automation/Manual Test Engineers describe this situation as a bug.

- A bug once detected can be reproduced with the help of standard bug-reporting templates.
- Major bugs are treated as prioritized and urgent especially when there is a risk of user dissatisfaction.
- The most common type of bug is a crash.
- Typos are also bugs that seem tiny but are capable of creating disastrous results.

❑ What is a Defect:

A defect refers to a situation when the application is not working as per the requirement and the actual and expected result of the application or software are not in sync with each other.

- The defect is an issue in application coding that can affect the whole program.
- It represents the efficiency and inability of the application to meet the criteria and prevent the software from performing the desired work.
- The defect can arise when a developer makes major or minor mistakes during the development phase.

❑ What is an Error:

Error is a situation that happens when the Development team or the developer fails to understand a requirement definition and hence that misunderstanding gets translated into buggy code. This situation is referred to as an Error and is mainly a term coined by the developers.

- Errors are generated due to wrong logic, syntax, or loop that can impact the end-user experience.
- It is calculated by differentiating between the expected results and the actual results.
- It raises due to several reasons like design issues, coding issues, or system specification issues and leads to issues in the application.

❑ What is a Fault:

Sometimes due to certain factors such as Lack of resources or not following proper steps Fault occurs in software which means that the logic was not incorporated to handle the errors in the application. This is an undesirable situation, but it mainly happens due to invalid documented steps or a lack of data definitions.

- It is an unintended behavior by an application program.
- It causes a warning in the program.
- If a fault is left untreated it may lead to failure in the working of the deployed code.
- A minor fault in some cases may lead to high-end error.
- There are several ways to prevent faults like adopting programming techniques, development methodologies, peer review, and code analysis.

❑ What is a Failure:

Failure is the accumulation of several defects that ultimately lead to Software failure and results in the loss of information in critical modules thereby making the system unresponsive. Generally, such situations happen very rarely because before releasing a product all possible scenarios and test cases for the code are simulated. Failure is detected by end-users once they face a particular issue in the software.

- Failure can happen due to human errors or can also be caused intentionally in the system by an individual.
- It is a term that comes after the production stage of the software.
- It can be identified in the application when the defective part is executed.



❖ Goals of Software Testing:

The main goal of Software Testing is to find bugs as early as possible, fix them, and deliver a fast, user-friendly product that meets both functional and non-functional requirements of the application while minimizing risks and costs.

Immediate Goals

- ☐ Bug Discovery

Short-Term Goals

- ☐ Reliability
- ☐ Quality
- ☐ Customer Satisfaction
- ☐ Risk Management

Post- Implementation Goals

- ☐ Reduce Maintenance Cost
- ☐ Improved Test Process
- ☐ Bug Prevention

1. Immediate Goals:

Immediate goals focus on the direct outcomes of testing. These objectives may be set at any time during the SDLC process which aims to catch the issues early and prevent the sources.

Some of these are covered in detail below:

1.Bug Discovery: This is the immediate goal of software testing to find errors at any stage of software development. The number of bugs is discovered in the early stage of testing. The primary purpose of software testing is to detect flaws at any step of the development process. The higher the number of issues detected at an early stage, the higher the software testing success rate.

2.Bug Prevention: This is the immediate action of bug discovery, that occurs as a result of bug discovery. Everyone in the software development team learns how to code from the behavior and analysis of issues detected, ensuring that bugs are not duplicated in subsequent phases or future projects.

2. Long-Term Goals:

These objectives have an impact on product quality in the long run after one cycle of the SDLC is completed. Long-term goals focus on improving product quality and user satisfaction over time, ensuring the software remains competitive and reliable after completing one SDLC cycle.

Some of these are covered in detail below:

1. **Quality:** This goal enhances the quality of the software product. Because software is also a product, the user's priority is its quality. Superior quality is ensured by thorough testing. Correctness, integrity, efficiency, and reliability are all aspects that influence quality. To attain quality, you must achieve all of the above-mentioned quality characteristics.
2. **Customer Satisfaction:** This goal verifies the customer's satisfaction with a developed software product. The primary purpose of software testing, from the user's standpoint, is customer satisfaction. Testing should be extensive and thorough if we want the client and customer to be happy with the software product.

2. Long-Term Goals:

These objectives have an impact on product quality in the long run after one cycle of the SDLC is completed. Long-term goals focus on improving product quality and user satisfaction over time, ensuring the software remains competitive and reliable after completing one SDLC cycle.

Some of these are covered in detail below:

3. **Reliability:** It is a matter of confidence that the software will not fail. In short, reliability means gaining the confidence of the customers by providing them with a quality product.
4. **Risk Management:** Risk is the probability of occurrence of uncertain events in the organization and the potential loss that could result in negative consequences. Risk management must be done to reduce the failure of the product and to manage risk in different situations.

3. Post-Implemented Goals:

Post-implementation goals focus on the software's performance after deployment and improving testing processes for future perspective.

Some of these are covered in detail below:

1. **Reduce Maintenance Cost:** Post-released errors are costlier to fix and difficult to identify. Because effective software does not wear out, the maintenance cost of any software product is not the same as the physical cost. The failure of a software product due to faults is the only expense of maintenance. Because they are difficult to discover, post-release mistakes always cost more to rectify. As a result, if testing is done thoroughly and effectively, the risk of failure is lowered, and maintenance costs are reduced as a result.
2. **Improved Software Testing Process:** These goals improve the testing process for future use or software projects. These goals are known as post-implementation goals. A project's testing procedure may not be completely successful, and there may be room for improvement. As a result, the bug history and post-implementation results can be evaluated to identify stumbling blocks in the current testing process that can be avoided in future projects.

❖ Central Issue in Testing:

The central issue in software testing revolves around the impossibility of exhaustive testing and the subsequent challenge of achieving adequate test coverage within practical constraints.

- ❑ **Infinite Test Cases:** Software applications, even moderately complex ones, possess an astronomically large number of possible inputs, states, and execution paths. It is practically impossible to test every single combination and scenario.
- ❑ **Resource Constraints:** Time, budget, and personnel limitations prevent testers from attempting to cover every conceivable test case. Decisions must be made about where to focus testing efforts.
- ❑ **Balancing Risk and Coverage:** The core challenge becomes how to select a subset of test cases that provides sufficient confidence in the software's quality and functionality, while managing the inherent risks of not testing every possible path. This involves prioritizing tests based on criticality, frequency of use, and potential impact of defects.
- ❑ **Evolving Software:** Software is rarely static. Changes, updates, and new features necessitate continuous testing, including regression testing, which adds another layer of complexity to maintaining adequate coverage.

❖ Testing Activities:

Software testing encompasses a range of activities aimed at ensuring the quality, functionality, and performance of a software product. These activities are typically structured within a Software Testing Life Cycle (STLC) and can be broadly categorized as follows:

1. Test Planning:

- ✓ **Requirement Analysis:** Understanding the software requirements to identify what needs to be tested.
- ✓ **Test Strategy & Plan Development:** Defining the overall approach to testing, including scope, objectives, resources, schedule, and types of testing to be performed.
- ✓ **Test Environment Setup:** Preparing the necessary hardware, software, and data for testing.

2. Test Design & Development:

- ✓ **Test Case Development:** Creating detailed test cases based on requirements, outlining steps, expected results, and test data.
- ✓ **Test Script Development:** For automated testing, writing code to automate test execution.

❖ Testing Activities:

3. Test Execution:

Test Execution: Running the developed test cases or scripts in the designated test environment.

Defect Logging & Reporting: Documenting any deviations from expected behavior as defects, providing clear details for developers.

Retesting & Regression Testing: Retesting fixed defects and performing regression tests to ensure that changes have not introduced new issues.

4. Test Monitoring & Control:

Progress Tracking: Monitoring the testing progress against the test plan.

Risk Management: Identifying and mitigating potential risks associated with testing.

Reporting: Providing regular updates on testing status and defect metrics to stakeholders.

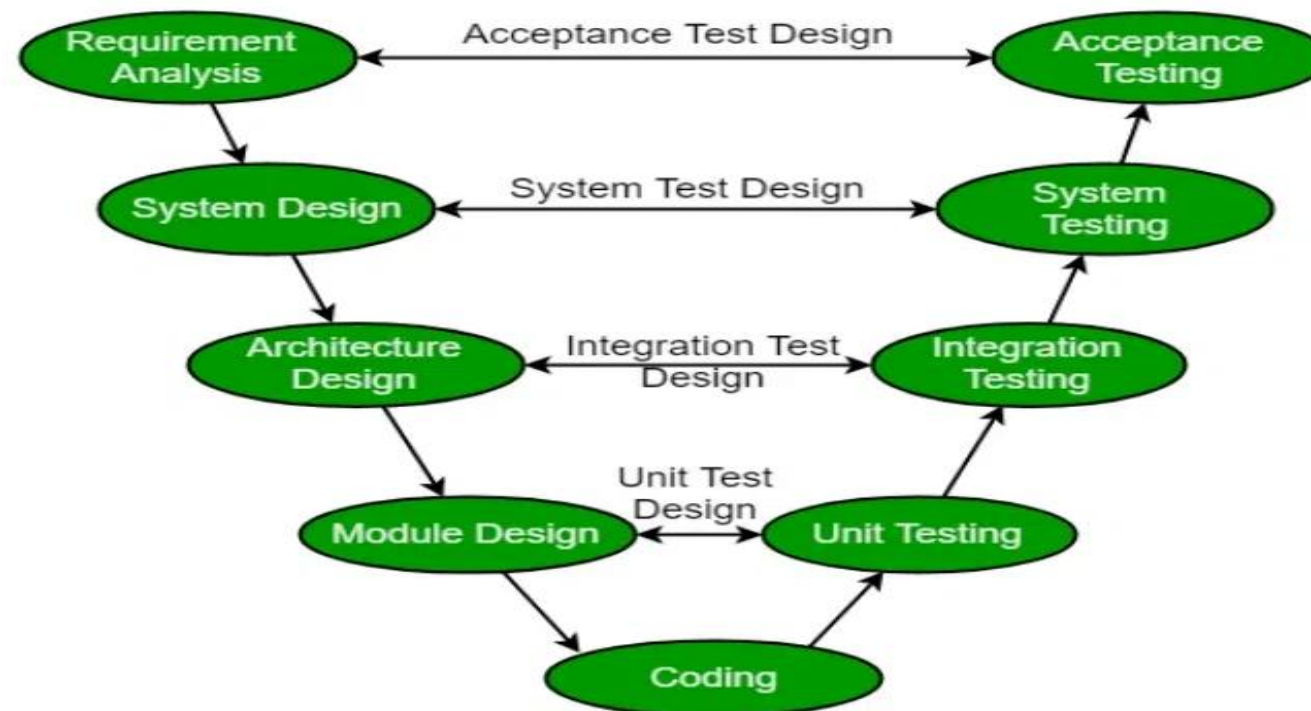
5. Test Completion:

Test Closure Activities: Finalizing testing, including preparing a test summary report, ensuring all defects are tracked, and archiving test artifacts.

Lessons Learned: Conducting a retrospective to identify areas for improvement in future testing efforts.

❖ V-Model:

V-Model is a type of Software Development Life Cycle (SDLC). It is a method that includes testing and validation alongside each development phase. It creates a structure like the letter 'V,' which includes various phases that we will discuss in detail.



❖ V-Model:

V-Model is a type of Software Development Life Cycle (SDLC). It is a method that includes testing and validation alongside each development phase. It creates a structure like the letter 'V,' which includes various phases that we will discuss in detail.

□ V-Model Verification Phases:

This is where the process begins. The first step is to gather and understand the customer's needs for the software. The goal is to define the scope of the project clearly to make sure everyone is on the same page.

It involves a static analysis technique (review) done without executing code. It is the process of evaluation of the product development phase to find whether specified requirements are met.

There are several Verification phases in the V-Model:

□ V-Model Verification Phases:

1. Business Requirement Analysis:

This is the first step of the designation of the development cycle where product requirement needs to be cured from the customer's perspective. These phases include proper communication with the customer to understand their requirements.

These are the very important activities that need to be handled properly, as most of the time customers do not know exactly what they want, and they are not sure about it at that time then we use an acceptance test design planning which is done at the time of business requirement it will be used as an input for acceptance testing.

❑ V-Model Verification Phases:

2. System Design:

In this phase, the overall structure of the software is planned out. The team develops both the high-level design (how the system will be structured) and detailed design (how the individual components will work).

Design of the system will start when the overall we are clear with the product requirements, and then need to design the system completely. This understanding will be at the beginning of complete under the product development process. these will be beneficial for the future execution of test cases.

❑ V-Model Verification Phases:

3. Architectural Design:

In this stage, architectural specifications are comprehended and designed. Usually, several technical approaches are put out, and the ultimate choice is made after considering both the technical and financial viability. The system architecture is further divided into modules that each handle a distinct function. Another name for this is High-Level Design (HLD).

At this point, the exchange of data and communication between the internal modules and external systems are well understood and defined. During this phase, integration tests can be created and documented using the information provided.

❑ V-Model Verification Phases:

4. Module Design:

This phase, known as Low-Level Design (LLD), specifies the comprehensive internal design for every system module. Compatibility between the design and other external systems as well as other modules in the system architecture is crucial.

Unit tests are a crucial component of any development process since they assist in identifying and eradicating the majority of mistakes and flaws at an early stage. Based on the internal module designs, these unit tests may now be created.

❑ V-Model Verification Phases:

5. Coding Phase:

This is where the software is actually built. Developers write the code based on the design created in the previous phase.

The Coding step involves writing the code for the system modules that were created during the Design phase. The system and architectural requirements are used to determine which programming language is most appropriate.

The coding standards and principles are followed when performing the coding. Before the final build is checked into the repository, the code undergoes many code reviews and is optimized for optimal performance.

□ V-Model Validation Phases:

It involves dynamic analysis techniques (functional, and non-functional), and testing done by executing code.

Validation is the process of evaluating the software after the completion of the development phase to determine whether the software meets the customer's expectations and requirements.

❑ V-Model Validation Phases:

1. Unit Testing:

In Unit testing, unit Test Plans are developed during the module design phase. These Unit Test Plans are executed to eliminate bugs in code or unit level.

2. Integration testing:

After completion of unit testing Integration testing is performed. In integration testing, the modules are integrated and the system is tested. Integration testing is performed in the Architecture design phase. This test verifies the communication of modules among themselves.

❑ V-Model Validation Phases:

3. System Testing:

System testing tests the complete application with its functionality, inter-dependency, and communication. It tests the functional and non-functional requirements of the developed application.

4. User Acceptance Testing (UAT):

User Acceptance Testing (UAT) is performed in a user environment that resembles the production environment. UAT verifies that the delivered system meets the user's requirement and the system is ready for use in the real world.

❖ **Monitoring and Measuring Test Execution:**

Test Execution is the process of executing the tests written by the tester to verify that the developed code, functions, or modules provide the expected result as per the client's or business's requirements. Test Execution comes under one of the phases of the Software Testing Life Cycle (STLC).

Test monitoring and measuring involves several core activities throughout the testing lifecycle:

- ✓ **Planning:** Defining clear objectives and key performance indicators (KPIs) for testing activities.
- ✓ **Data Collection:** Gathering relevant data points during the test execution phase, often through test management and automation tools.
- ✓ **Data Analysis:** Analyzing collected data to identify trends, patterns, and deviations from the original plan.

❖ **Monitoring and Measuring Test Execution:**

- ❑ Test Execution is the process of executing the tests written by the tester to verify that the developed code, functions, or modules provide the expected result as per the client's or business's requirements. Test Execution comes under one of the phases of the Software Testing Life Cycle (STLC).
- ❑ In software testing, monitoring and measuring test execution involves continuously tracking, analyzing, and reporting key metrics to assess progress, quality, and efficiency, which in turn enables informed decision-making and corrective actions

❑ Test Execution For Software Testing:

Test Execution is the process of executing the tests written by the tester to verify that the developed code, functions, or modules provide the expected result as per the client's or business's requirements. Test Execution comes under one of the phases of the Software Testing Life Cycle (STLC).

❑ Process of Test Execution:

- ✓ The test Execution technique consists of three different phases, which will be carried out to process the test result and ensure the correctness of the required results.
- ✓ In each phase, various activities or work will be carried out by various team members. The three main phases of test execution are the creation of test cases, test case execution, and validation of test results. Let us discuss each phase.

□ Process of Test Execution:

1. Creation of Test Cases:

The first phase is to create suitable test cases for each module or function. Here, the tester with good domain knowledge must be required to create suitable test cases. It is always preferable to create simple test cases and the creation of test cases should not be delayed else it will cause excess time to release the product.

The created test cases should not be repeated again. It should cover all the possible scenarios raised in the application.

2. Test Cases Execution

After test cases have been created, execution of test cases will take place. Here, the Quality Analyst team will either do automated or manual testing depending upon the test case scenario.

It is always preferable to do both automated as well as manual testing to have 100% assurance of correctness. The selection of testing tools is also important to execute the test cases.

□ Process of Test Execution:

3. Validating Test Results

After executing the test cases, note down the results of each test case in a separate file or report. Check whether the executed test cases achieved the expected result and record the time required to complete each test case i.e., measure the performance of each test case.

If any of the test cases is failed or not satisfied the condition then report it to the development team for validating the code.

□ Ways to Perform Test Execution:

Testers can choose from the below list of preferred methods to carry out test execution:

- **Run test cases:** It is a simple and easiest approach to run test cases on the local machine and it can be coupled with other artifacts like test plans, test suites, test environments, etc.
- **Run test suites:** A test suite is a collection of manual and automated test cases and the test cases can be executed sequentially or in parallel. Sequential execution is useful in cases where the result of the last test case depends on the success of the current test case.
- **Run test case execution and test suite execution records:** Recording test case execution and test suite execution is a key activity in the test process and helps to reduce errors, making the testing process more efficient.
- **Generate test results without execution:** Generating test results from non-executed test cases can be helpful in achieving comprehensive test coverage.

□ Ways to Perform Test Execution:

- **Modify execution variables:** Execution variables can be modified in the test scripts for particular test runs.
- **Run automated and manual tests:** Test execution can be done manually or can be automated.
- **Schedule test artifacts:** Test artifacts include video, screenshots, data reports, etc. These are very helpful as they document the results of the past test execution and provide information about what needs to be done in future test execution.
- **Defect tracking:** Without defect tracking test execution is not possible, as during testing one should be able to track the defects and identify what when wrong and where.

❖ **Monitoring and Measuring Test Execution:**

Test monitoring and measuring involves several core activities throughout the testing lifecycle:

- ✓ **Planning:** Defining clear objectives and key performance indicators (KPIs) for testing activities.
- ✓ **Data Collection:** Gathering relevant data points during the test execution phase, often through test management and automation tools.
- ✓ **Data Analysis:** Analyzing collected data to identify trends, patterns, and deviations from the original plan.
- ✓ **Reporting:** Communicating the status and insights to stakeholders using dashboards and reports to ensure transparency.
- ✓ **Test Control:** Implementing corrective actions (like adjusting resources or revising schedules) based on the analysis to keep the project on track.

❖ Automation Testing:

- Automated Testing means using special software for tasks that people usually do when checking and testing a software product. Nowadays, many software projects use automation testing from start to end, especially in agile and DevOps methods.
- This means the engineering team runs tests automatically with the help of software tools. It will help the testing team to make the process faster. Continuous delivery (CD) quickly sends the new code to users.
- Automated testing is important for this because it converts the manual steps into automation. Continuous integration (CI) checks the new code changes to prevent issues. Automated testing, CI & CD will together prove that the new code is error-free and ready for deployment quickly for the project's purpose.

❖ Automation Testing:

Automation Testing is a technique where the Tester writes scripts independently and uses suitable Software or Automation Tools to test the software. It is an Automation Process of a Manual Process. It allows for executing repetitive tasks without the intervention of a Manual Tester.

- ✓ It is used to automate the testing tasks that are difficult to perform manually.
- ✓ Automation tests can be run at any time of the day as they use scripted sequences to examine the software.
- ✓ Automation tests can also enter test data, compare the expected result with the actual result and generate detailed test reports.
- ✓ The goal of automation tests is to reduce the number of test cases to be executed manually but not to eliminate manual testing.
- ✓ It is possible to record the test suite and replay it when required.

❖ Automation Testing:

Automated Testing uses specialized software to replace manual testing tasks, speeding up the process and integrating seamlessly with CI/CD pipelines.

It allows for continuous code verification and quicker deployment.

❑ Why Transform From Manual to Automated Testing?:

In 1994, an aircraft completing its routine flight crashed just before landing. This was due to a bug or defect in their software.

The testers didn't even care about the final testing, so this accident happened.

In such high-priority cases, accurate automation testing becomes critical to replace a few manual tests.

Below are some of the reasons for using automation testing:

- **Quality Assurance:** Manual testing is a tedious task that can be boring and at the same time error-prone. Thus, using automation testing improves the quality of the software under test as more test coverage can be achieved.

❑ Why Transform From Manual to Automated Testing?:

- **Quality Assurance:** Manual testing is a tedious task that can be boring and at the same time error-prone. Thus, using automation testing improves the quality of the software under test as more test coverage can be achieved.
- **Error or Bug-free Software:** Automation testing is more efficient for detecting bugs in comparison to manual testing.
- **No Human Intervention:** Manual testing requires huge manpower in comparison to automation testing which requires no human intervention and the test cases can be executed unattended.
- **Increased test coverage:** Automation testing ensures more test coverage in comparison to manual testing where it is not possible to achieve 100% test coverage.
- **Testing can be done frequently:** Automation testing means that the testing can be done frequently thus improving the overall quality of the software under test.

❑ Automation Testing Types:

Below are the different types of automation testing:

- **Unit testing:** Unit testing is a phase in software testing to test the smallest piece of code known as a unit that can be logically isolated from the code. It is carried out during the development of the application.
- **Integration testing:** Integration testing is a phase in software testing in which individual software components are combined and tested as a group. It is carried out to check the compatibility of the component with the specified functional requirements.
- **Smoke testing:** Smoke testing is a type of software testing that determines whether the built software is stable or not. It is the preliminary check of the software before its release in the market.

❑ Automation Testing Types:

- **Performance testing:** Performance testing is a type of software testing that is carried out to determine how the system performs in terms of stability and responsiveness under a particular load.
- **Regression testing:** Regression testing is a type of software testing that confirms that previously developed software still works fine after the change and that the change has not adversely affected existing features.
- **Security testing:** Security testing is a type of software testing that uncovers the risks, and vulnerabilities in the security mechanism of the software application. It helps an organization to identify the loopholes in the security mechanism and take corrective measures to rectify the security gaps.

❑ Automation Testing Types:

- **Acceptance testing:** Acceptance testing is the last phase of software testing that is performed after the system testing. It helps to determine to what degree the application meets end users' approval.
- **API testing:** API testing is a type of software testing that validates the Application Programming Interface(API) and checks the functionality, security, and reliability of the programming interface.
- **UI Testing:** UI testing is a type of software testing that helps testers ensure that all the fields, buttons, and other items on the screen function as desired.

❑ Popular Automation Tools:

- **Selenium:** Selenium is an automated testing tool that is used for Regression testing and provides a playback and recording facility. It can be used with frameworks like JUnit and Test-NG. It provides a single interface and lets users write test cases in languages like Ruby, Java, Python, etc.
- **QTP:** Quick Test Professional (QTP) is an automated functional testing tool to test both web and desktop applications. It is based on the VB scripting language and it provides functional and regression test automation for software applications.
- **Sikuli:** It is a GUI-based test automation tool that is used for interacting with elements of web pages. It is used to search and automate graphical user interfaces using screenshots.
- **Appium:** Appium is an open-source test automation framework that allows QAs to conduct automated app testing on different platforms like iOS, Android, and Windows SDK.
- **Jmeter:** Apache JMeter is an open-source Java application that is used to load test the functional behavior of the application and measure the performance.

❑ Automation Testing Types:

- **Acceptance testing:** Acceptance testing is the last phase of software testing that is performed after the system testing. It helps to determine to what degree the application meets end users' approval.
- **API testing:** API testing is a type of software testing that validates the Application Programming Interface(API) and checks the functionality, security, and reliability of the programming interface.
- **UI Testing:** UI testing is a type of software testing that helps testers ensure that all the fields, buttons, and other items on the screen function as desired.

❖ Limitations of Testing:

- There are certain limitations of testing in software testing that need to be considered such as incomplete test coverage, the presence of undetectable errors, time and resource constraints, reliance on test data, and the inability to guarantee absolute correctness.
- It's important to acknowledge these limitations and implement strategies to mitigate their impact during the testing process.
- Key Limitations of Testing in Software Development:

❖ Limitations of Testing:

- **Testing cannot typically uncover things that are missing or unknown:**

Tests can be written to find and identify known issues. Still, if there are issues that aren't well understood, there's no way to design an appropriate test to find them. Therefore, testing simply cannot guarantee that an application is free from errors.

- **It's impossible to test for all conditions:**

There is effectively an infinite number and variety of inputs that can go into an application, and testing them all, even if theoretically possible, is never going to be practical. Testing must cover the known probable inputs, and this is a balance between quality and deadlines.

❖ Limitations of Testing:

- Testing usually gives no insight into the root causes of errors:
These causes need to be identified to prevent repeating the same mistakes in the future. Yet, testing will only tell whether or not the issues are present.
- Time and resource constraints:
Limited time and resources may restrict the extent of testing that can be performed.
- Reliance on test data:
Testing relies on the availability of representative and comprehensive test data for accurate results.

❖ Limitations of Testing:

- Inability to guarantee absolute correctness:

Testing can provide confidence in the software's functionality, but it cannot guarantee that it is entirely error-free.

In summary, the limitations of testing in software testing come from practical, financial, and time restrictions. The result of these restrictions is a set of hard truths that testing cannot be exhaustive, can only be completed to a particular budget, and will always be a compromise between the release date and the quality of software.

Thank You